**CSCE 411: Design and Analysis of Algorithms**

# Week 10: Computational Complexity

*Date: March 24, 2026* *Nate Veldt, updated by Samson Zhou*

**Course Logistics**

- Exam 2 next Thursday
- Review session next Tuesday

## 1 The universe of computational problems

**Refresher on computational problems** It will be important for us to distinguish between three concepts in defining and solving computational problems:

1. Problem: a computational task that takes in an input and seeks a specific output.

2. Instance: one specific input for the problem

3. Candidate solution: the data encoding a possible solution to an instance.

## 1.1 Decision Problems and Optimization Problems

A computational problem $Q$ is a *decision* problem if

An optimization problem is a problem that seeks the minimum or maximum value of a function subject to constraints.

**Question 1.** *Which of the following is a decision problem?*

A *Find the maximum flow for a graph $G = (V, E)$ with source/sink nodes $s$ and $t$.*

B *Find if the graph $G = (V, E)$ is connected.*

C *Find the shortest path of a graph $G = (V, E)$*

D *Find if $G = (V, E)$ is a directed acyclic graph*

E *More than one of the above*

For any optimization problem, we can define a corresponding *decision* problem by taking an extra input $k$ and asking:

## 1.2 The problem class P

A problem $Q$ is said to be in $P$ if there is a polynomial time algorithm that solves the problem.

In other words, if $n$ is the size of the input data, the problem has a runtime such as:

These are often referred to as _____ problems and are treated in theory as problems that can be solved _____.

All of the problems we have considered so far are polynomial time, but this is only a small sample of problems.

## 1.3 The problem class NP

We'll define NP problems using a few different levels of technical difficulty.

**Level 1: informal, gets us most of the way there.** A problem is in NP if we can check in polynomial time whether a given candidate solution solves the instance or not.

**Level 2: certificates and verifiers.** NP is the set of decision problems with the following property:

If the answer to an instance is *yes*, there exists some data called a *certificate* with which an algorithm can *verify* the answer is YES in polynomial time.

The *certificate* is

The *verifier* is

**Level 3: the technically precise definition.** NP is the set of decision problems that can be solved by

This involves defining Turing machines, formal languages, and a whole lot more.

You do not need to process this definition (or all the textbook details) for this course. But it does tell us where the name "NP" comes from:

**Examples** A problem is in NP if for every YES instance there is a *certificate* that an algorithm can use to *verify* in polynomial time that it is truly a YES instance.

**Example 1.** Does $G = (V, E)$ have a clique of size at least $k$?

**Certificate**

**Verifier**

**Example 2.** Is $G = (V, E)$ a connected graph?

**Certificate**

**Verifier**

**Proving something is in NP:** *If you can check in polynomial time whether a candidate solution solves the problem or not, then the problem is in NP.*

## 1.4 The co-NP class

A problem is in co-NP if for every _____ there is a *certificate* (some data) that an algorithm can use to *verify* in polynomial time that it is truly a _____

To be clear, we will often use the terms _____-certificate and _____-verifier.

**Example problem** Is $G = (V, E)$ a directed acyclic graph?

**Certificate**

**Verifier**

**Conclusion**: Checking whether $G$ is a DAG is _____

**Question 2.** *The verifier we showed for the clique problem will return YES if the $k$ nodes are a clique, and will return NO if the $k$ nodes are not a clique.*

*True or false: this means that this algorithm is also a NO-verifier, and so the clique problem is also in co-NP.*

A *True*

B *False*

**Question 3.** *Checking whether $G$ is a DAG in in co-NP. Is it also in NP?*

A *Yes*

B *No*

## 2 Reductions

A reduction is a mapping from one computational problem to another.

**Definition** Let $A$ and $B$ be decision problems. We say that $A$ can be reduced to $B$ if

- For every instance of $A$ we can define a corresponding instance of $B$ such that
- All YES instances in $A$ map to YES instances in $B$
- All NO instances in $A$ map to NO instances in $B$

This is a polynomial-time reduction if this conversion process takes polynomial time.

This seems obvious: what's an example of a reduction that is not polynomial-time?

## 2.1 Reduction and complexity

If $A$ can be reduced to $B$ in polynomial time, we denote this by

_____

This implies that:

- $A$ is easier[1] than $B$, or equivalently
- $B$ is at least as hard as $A$ is to solve

**Lemma 2.1.** *If $B \in P$ and* _____*, then* _____

[1] where easier might mean "no harder than"

## 3 NP-completeness

**Definition:** A problem $Q$ is NP-complete if:

1. $Q \in \text{NP}$
2. for every problem $B \in \text{NP}$, $B \leq_p Q$.

**In words:**

This implies that NP-complete problems are the *hardest* problems in NP.

Why? Remember that $A \leq_p B$ means $A$ is easier than $B$.

In fact, if you only take the second part of the definition, this defines the set of NP-hard problems.

## 4 Reductions

A reduction is a mapping from one computational problem to another.

**Definition** Let $A$ and $B$ be decision problems. We say that $A$ can be reduced to $B$ if

- For every instance of $A$ we can define a corresponding instance of $B$ such that
- All YES instances in $A$ map to YES instances in $B$
- All NO instances in $A$ map to NO instances in $B$

This is a polynomial-time reduction if this conversion process takes polynomial time.

This seems obvious: what's an example of a reduction that is not polynomial-time?

## 4.1 Reduction and complexity

If $A$ can be reduced to $B$ in polynomial time, we denote this by

_____

This implies that:

- $A$ is easier[2] than $B$, or equivalently
- $B$ is at least as hard as $A$ is to solve

**Lemma 4.1.** *If $B \in P$ and* _____*, then* _____

[2] where easier might mean "no harder than"

## 5 NP-completeness

**Definition:** A problem $Q$ is NP-complete if:

1. $Q \in \text{NP}$
2. for every problem $B \in \text{NP}$, $B \leq_p Q$.

**In words:**

This implies that NP-complete problems are the *hardest* problems in NP.

Why? Remember that $A \leq_p B$ means $A$ is easier than $B$.

In fact, if you only take the second part of the definition, this defines the set of NP-hard problems.

## 6 Views of the computational problem universe

There are four different possibilities for the relationship between P, NP, and co-NP.

There are two possibilities for the relationship between P, NP, and NP-complete.

**A few famous NP-complete problems**

- Find if a graph has a clique of size at least $k$
- Find if a graph has a Hamiltonian cycle
- Find if a graph has a simple path with at least $k$ edges
- **Subset sum**: given a set of numbers, find if any subset sums to zero
- **SAT**: Logic problems we will look at in more depth later

## 7 Summary on Reduction and NP-completeness

If $A$ can be reduced in polynomial time to $B$, we write $A \leq_p B$. This means that $A$ is no harder than B; a solver for $B$ can be used to solve $A$.

A problem $Q$ is NP-hard if every problem in NP can be reduced to $Q$ in polynomial time.

A problem $Q$ is NP-complete if

- It is in NP
- It is NP-hard

**Question 4.** *Assume that $A$ and $B$ are both in NP and $A \leq_p B$. Which of the following statements is false?*

A *If $A \in P$, then $B \in P$*

B *If $A$ is NP-complete, then $B$ is NP-complete*

C *If $B \in P$, then $A \in P$*

D *If $A$ is NP-complete, then $B \leq_p A$*

If two problems are NP-complete, then they are both in NP, and:

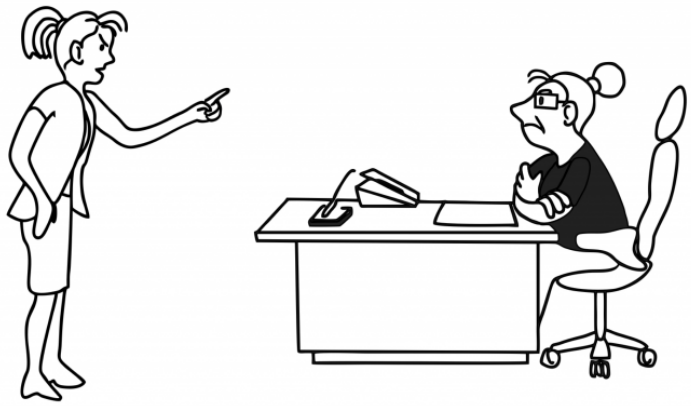
## 8 Why does it matter whether a problem is in P, NP, or is NP-complete?

A famous book by Garey and Johnson, *Computers and Intractability*, includes some famous cartoons to help us answer this question.[3]

Let's say your boss tells you that because you are a CS major or minor you should be able to find an efficient algorithm for a certain problem at work. You work for weeks without progress. You don't want to tell your boss:

**"I can't find an efficient algorithm, I guess I'm just too dumb."**

Ideally you'd like to be able to say:



**"I can't find an efficient algorithm, because no such algorithm is possible!"**

But in most situations, proving definitively that this is impossible is unlikely.

[3] Garey, Michael R., and David S. Johnson. Computers and intractability. Vol. 174. San Francisco: freeman, 1979. High-resolution versions of the cartoon are available online.

If you fail to find an efficient solution to a problem, ask yourself: am I struggling to find a solution simply because this is NP-complete? If you can prove it is, you have a great answer for your boss.

**"I can't find an efficient algorithm, but neither can all these famous people."**

## 9 A brief and incomplete history of NP-completeness

- 1971: Stephen Cook proves that every problem in NP can be reduced to a problem called *Boolean Satisfiability* (SAT).

  In modern terminology: SAT is NP-complete.

  The same results were developed independently by someone named Leonid Levin, so this is now called the *Cook-Levin* Theorem.

- 1972: Richard Karp publishes a paper *Reducibility Among Combinatorial Problems*, showing that SAT can be reduced to 21 other problems.

- 1979: Garey and Johnson publish the go-to reference on NP-completeness: *Computers and Intractability.*